

From Transformer RTL to an Inference Product

A practical implementation roadmap for real models, reusable hardware, and manufactured silicon

Starting point

The local verilog project contains a frozen-weight Q8 tile, a CPU reference, and a small RTL decoder prototype. This handbook identifies what is missing, what to implement, what to purchase or obtain, and what evidence is required before claiming a working product.

The execution requirement

All transformer forward-pass tensors must be computed by the accelerator. Host software may prepare models, tokenize text, manage requests, and independently verify outputs. A CPU numerical result must not silently substitute for missing hardware inference.

Version 1.0 21 September 2026

Repository: `/Users/computer/dev/verilog`

Scope: inference only; fixed-model and programmable ASIC choices; large-model scaling.

Status: planning document, not a tapeout package or a fabrication quotation.

How to use this handbook

Start with Sections 1–3. They define the actual starting point and prevent the project from expanding into an undefined promise to run every possible model. Then work through the implementation gates in Section 20. The checklists describe work packages, not features that already exist.

Recommended direction for your stated goal: use reloadable weights, a programmable execution schedule, and a declared operator set. Develop the correctness and memory interfaces first. Treat a permanently fused model as a separate specialization after the model and its commercial lifetime are known.

Read the status labels literally

Label	What it establishes
Implemented proto-type	Source exists and some behavior is exercised; correctness or scale may remain unqualified.
Verified component	Defined tests or proofs pass for a stated configuration and numerical contract.
Mapped hardware	Arithmetic and memories have been implemented using actual target resources.
Physical sign-off	Required geometry, connectivity, timing and electrical checks pass for the target flow.
Product qualified	Packaged hardware, software, manufacturing test and required qualification have passed.

Three rules for reading the estimates

1. Capacity and bandwidth examples are calculations with stated assumptions; they are not measured performance predictions.
2. An arbitrary Verilog array does not establish a feasible physical memory. Its ports, access latency, banking and electrical behavior must match a real implementation.
3. No finite chip supports arbitrary future architectures. Publish supported operators, numerical formats, shapes and capacity limits, then reject unsupported models.

Scope and evidence

The repository observations refer to the files present on 21 September 2026. Previously recorded experiments are identified as such; this document does not claim a new full-model inference or physical-design run. External references are collected at the end. Foundry, supplier, licensing and regulatory details must be confirmed for the product actually selected.

Contents

1	Current project: what works and what is still unproven	4
2	Choose the product and the model-update strategy	5
3	Define the execution and numerical contract	6
4	Repair and qualify the current arithmetic and protocols	7
5	Build the model importer and compiler	8
6	Implement reusable matrix and vector engines	9
7	Complete the transformer graph and attention	10
8	Engineer weight memory, scratchpads and data movement	11
9	Size model storage, KV capacity and bandwidth	12
10	Manage KV state, requests and long contexts	13
11	Make execution programmable and observable	14
12	Support different families and very large models	15
13	Build independent verification and release evidence	16
14	Model performance and validate on a hardware prototype	17
15	Integrate the chip platform and obtain technology IP	18
16	Close physical design and prepare a tapeout package	19
17	Design production test and silicon reliability	20
18	Build the package, board, cooling and service software	21
19	Resolve commercial, licensing and compliance work	22
20	Execute the project in evidence-based stages	23
21	Maintain a release checklist and risk register	24
22	Repository map and commands	25
23	Glossary	26
	References and source boundaries	27

1 Current project: what works and what is still unproven

Asset	Actual scope
Frozen model image	Local Llama 3.2 1B Instruct Q8_0/F32 tensor package: 1,313,251,456 bytes including alignment. It is a data image, not a complete chip.
Fixed-weight Q8 tile	Default co-simulation selects eight rows of the first query projection. The physical experiment uses one row. Neither executes a whole attention layer.
CPU reference	Complete reference execution; the hybrid path replaces selected CPU outputs with simulated tile outputs. It is not standalone accelerator inference.
New decoder RTL	Serial Q16.16 prototype containing the Llama-style graph, mutable parameter arrays, KV state, tied output projection and greedy argmax.
Decoder regression	Two layers, hidden width four, two heads, four vocabulary entries and two cached positions; synthetic weights and identity RoPE.
Decoder synthesis target	Generic lowering and structural checks; memories and arithmetic operators remain. No target-library timing or completed full gate mapping is established.
SKY130 experiment	A separate one-row tile reached placement/clock/global-route stages; no clean final routed implementation of the full decoder.

Do not carry these assumptions into a product plan

- A test that returns two expected token IDs does not verify every projection, normalization, probability, activation, or final logit.
- Increasing elaboration parameters does not prove memory capacity, routing, clock frequency or runtime model compatibility.
- Structural synthesis is not equivalent to a clean standard-cell netlist, physical sign-off, or a working fabricated chip.
- The older hardmax/ReLU tiny transformer and simulation-only GPT reference are different designs; their capabilities do not automatically belong to the new decoder.

Deliverable: A baseline evidence register identifying source revision, model hash, tool versions, tests and reports for each claim.

Completion evidence: Every status claim links to a reproducible artifact and lists its configuration and limits.

2 Choose the product and the model-update strategy

Architecture	Strength	Constraint
Fixed-model ASIC	Specializes weight storage and computation for a chosen checkpoint.	New base weights generally require different manufactured ROM/silicon; adapters only help if supported.
Programmable ASIC	Loads new weights and schedules supported operators on the same device.	Pays for writable memory, addressing, control and generality.
Hybrid	Combines fixed regions with programmable engines or writable overlays.	Only the deliberately implemented update path is available.

Taalas publicly describes specialization for individual models and integration of storage with computation [1]. Its exact implementation is not a public specification for this project. Do not assume that an ordinary synthesized ROM reproduces its physical density or performance.

Write a product requirements document

- ☐ Select a card, embedded module, appliance, or hosted service as the first deliverable.
- ☐ Identify initial checkpoints and supported architecture variants, including rights to use their weights.
- ☐ Set maximum parameters, layers, hidden/FFN widths, head counts, vocabulary and context.
- ☐ Set batch/concurrency limits, prompt/output length distributions and quality targets.
- ☐ Set time-to-first-token, inter-token latency and throughput targets, including p95/p99 behavior where relevant.
- ☐ Set board power, thermal envelope, dimensions, environmental conditions and host-interface requirements.
- ☐ Set unit-cost target, volume assumptions, support lifetime and model-update policy.
- ☐ Name the reference systems and workloads used to decide whether the product is competitive.

Training and optimizer hardware are outside the initial scope. Loading weights trained elsewhere is sufficient for inference. Supporting adapters or fine-tuning does not require putting training on this accelerator.

Deliverable: Product requirements, a supported-model matrix and an architecture decision record.

Completion evidence: The first release has measurable limits; model updates and unsupported operators have explicit behavior.

3 Define the execution and numerical contract

The hardware and compiler must agree on exact tensor layouts, data formats, operation semantics and error behavior. A model name or an “INT8” label is not enough to establish compatibility.

- Define weight, activation, accumulator and KV formats separately: signedness, scale/exponent representation, group size, packing, endianness and alignment.
- Define scale/zero-point interpretation and support for mixed-precision tensors.
- Specify reduction order, intermediate widths, rounding, saturation, overflow and underflow; include FP special values wherever FP arithmetic is supported.
- Define supported shapes, tail masks, strides, transposes, broadcast rules and address widths.
- Specify nonlinear approximation domains, maximum error and out-of-domain behavior.
- Set normalization/Softmax precision independently of low-bit matrix weights.
- Define legal command sequences, reset/cancel behavior, model lifetime and cache validity.
- State which outputs the accelerator returns: logits, selected tokens, diagnostic tensors, or a supported combination.

Separate four meanings of “exact”

Property	Evidence needed
Graph equivalence	Same mathematical operators, dimensions, masks, positions and dependencies as the checkpoint.
Arithmetic equivalence	Same specified finite-precision operations and rounding as the chosen hardware oracle.
Checkpoint quality	Measured numerical and task-quality impact of quantization and approximation.
Product reliability	Correct execution under supported workloads, stalls, resets and operating conditions.

A lower-precision implementation can preserve the graph while producing different logits. An approximate exponential is not automatically acceptable because it is synthesizable. Choose tolerances before evaluating results.

Deliverable: A numerical specification and operator/interface conformance suite.

Completion evidence: Two independent implementations agree within the specified contract, including edge cases.

4 Repair and qualify the current arithmetic and protocols

Treat these as priority engineering tasks before adding larger matrices.

Current issue	Required action and test
Q16.16 resolution	Its LSB is $2^{-16} \approx 1.526 \times 10^{-5}$. Select a representation for the checkpoint's 10^{-5} RMS epsilon and qualify the resulting error.
Exponential clipping	The current function clips to $[-8, 8]$. Measure probability-tail and SiLU errors; replace it with a bounded-error implementation for the actual domain.
Softmax range	A Q16.16 sum saturates near 32,768 unit exponentials. Widen/rescale accumulators and prove supported context limits.
Premature saturation	Per-product/per-add saturation can destroy cancellation. Use justified accumulator width and test long mixed-sign reductions.
Signed arithmetic	Audit signed division, negation of the most-negative integer, casts and concatenations. Test positive/negative operands against an independent oracle.
Unpipelined operators	Division, square root and dependent multipliers need implementation latencies and timing constraints; source-level operators alone are insufficient.
Input lifetime	Latch token/command inputs when accepted or define and verify a stable-input contract. The present embedding reads the token over multiple cycles.
Context/model changes	Report full-context and invalid-token errors. Block incompatible model writes or invalidate KV state; define clear/start priority.
Small regression	Add nonidentity RoPE, signed/random tensors, intermediate checks and full-cache/reset/stall cases.

The current test exercises some nonzero arithmetic, but it mainly checks argmax, position and positivity. It does not establish full numeric correctness. No repairs to RTL are performed by producing this handbook.

Deliverable: A reviewed issue list, arithmetic units with a defined contract, and regression vectors.

Completion evidence: All listed behaviors have passing tests; unsupported ranges produce defined errors rather than silent wrong results.

5 Build the model importer and compiler

The importer translates a checkpoint into the accelerator's representation. The compiler maps its graph onto available compute and memory resources.

- Parse supported containers and configuration metadata; validate tensor count, names, shapes, data types and file bounds before allocation or execution.
- Handle tied/separate embedding and output weights, biases, norm constants, head grouping, tensor permutations and RoPE conventions.
- Decode supported quantization formats. Q8_0 consists of an FP16 block scale and 32 signed INT8 values; other "8-bit" formats are not necessarily interchangeable.
- Define whether to preserve Q8_0 or calibrate a new format. Expanding a previously quantized checkpoint does not recover its original lost precision.
- Pack weights, scales and constants into aligned banks; generate bank addresses and integrity hashes without silently dropping unsupported tensors.
- Allocate activation/residual lifetimes, scratch space and KV storage.
- Tile and fuse supported operations; schedule memory transfers, reductions and barriers.
- Emit instructions/descriptors plus a manifest containing dimensions, operator requirements, numerical contract, compiler version and model hash.
- Validate capacity and address calculations using widths large enough for the largest image; do not rely on signed 32-bit host/RTL intermediates for huge tensors.
- Provide explicit diagnostics for unsupported operations, formats or capacity.

Offline software is compatible with hardware inference

Python or C++ may prepare constants, including a qualified RoPE lookup table, and independently compute expected results. During an inference run, all transformer tensors must come from the hardware datapath. The runtime must not quietly route unsupported transformer operations to the CPU.

Deliverable: Importer, compiler, model-image format, loader compatibility rules and test corpus.

Completion evidence: A real checkpoint imports reproducibly; its generated schedule executes without host tensor substitution.

6 Implement reusable matrix and vector engines

Matrix engine

- Implement GEMV for incremental decode and GEMM for prompt processing/batching.
- Select lane count, multiplier precision, accumulator width, pipeline depth and reduction topology from bandwidth, area and performance budgets.
- Support programmable dimensions, strides, base addresses and matrix tiling.
- Handle partial tiles, ragged dimensions, transposes and masked tail lanes.
- Accept mutable activation operands for attention, not only fixed weight operands.
- Implement weight unpacking, block scales, quantization/dequantization and output conversion.
- Provide banked accumulators, input/output buffers and overlap of transfer with computation.
- Export completion, exceptions, utilization and stall counters.

Vector and reduction engine

- Implement add, multiply, bias, sum, maximum and sum-of-squares reductions.
- Implement RMSNorm; add LayerNorm only if included in the declared model set.
- Implement reciprocal/reciprocal-square-root, exponential, SiLU and any required GELU variant.
- Implement residuals, quantization, clipping and model-specific logit transforms where required.
- Give every multicycle unit an explicit request/response protocol and reset behavior.
- Size shared resources so normalization and Softmax do not stall a much larger matrix array indefinitely.

An RTL function call can instantiate substantial combinational hardware at each call site. Serial control does not automatically mean that synthesis shares a single physical arithmetic unit. Inspect the mapped structure and resource use.

Deliverable: Parameterized compute engines with latency-aware interfaces and a mapped representative tile.

Completion evidence: Operator accuracy, stall behavior, resource use and timing are measured for the selected target.

7 Complete the transformer graph and attention

For the Llama-style path, the per-token computation is:

$$\begin{aligned}
 n &= \text{RMSNorm}(x), & Q &= W_Q n, \quad K = W_K n, \quad V = W_V n, \\
 \widehat{Q} &= \text{RoPE}(Q), & \widehat{K} &= \text{RoPE}(K), \\
 P &= \text{softmax}\left(\frac{\widehat{Q} K_{\text{cache}}^T}{\sqrt{d_h}} + M\right), & r &= x + W_O(P V_{\text{cache}}), \\
 u &= \text{RMSNorm}(r), & y &= r + W_{\text{down}}[\text{SiLU}(W_{\text{gate}} u) \odot W_{\text{up}} u].
 \end{aligned}$$

The current token's rotated K and V must be appended before its attention reads them. M denotes the applicable mask; GQA maps query heads onto their shared KV heads. Norm weight vectors and epsilon are part of each normalization.

- ☐ Verify all Q/K/V/O dimensions and head permutations against checkpoint conventions.
- ☐ Support required position indices, rotary dimensions, frequencies and scaling.
- ☐ Implement the appropriate causal, padding and optional sliding-window masks.
- ☐ Compute QK scores, stable Softmax and probability-weighted V in the specified precision.
- ☐ Implement prompt and decode execution modes with consistent cache contents.
- ☐ Consider streaming/tiled Softmax to reduce attention intermediate storage; qualify its accumulation order and approximation error.
- ☐ Preserve residuals until consumed and schedule all layers without CPU math.
- ☐ Execute embedding lookup, final normalization and vocabulary projection.
- ☐ Support untied output weights where required; the present prototype assumes tied weights.

The prefill score domain grows with prompt length; incremental decode reads an existing cache. Their data reuse and scheduling differ. NVIDIA's documentation provides concrete examples of attention and cache-serving modes [2].

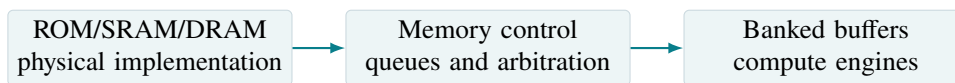
Deliverable: One full real-checkpoint layer, then the complete real decoder.

Completion evidence: Compare intermediate tensors and final logits across multiple tokens and nontrivial positions.

8 Engineer weight memory, scratchpads and data movement

- ❑ Budget weight bytes, scales, padding, protection, activations, residuals, accumulators, instructions and KV separately.
- ❑ Choose on-chip ROM/SRAM capacity and any external memory technology based on both sustainable bandwidth and storage demand.
- ❑ Define banks, ports, access width, latency, burst length and alignment.
- ❑ Obtain usable memory macro views: functional model, timing, physical abstract, layout, power and electrical information required by the flow.
- ❑ Implement a memory controller and its PHY for external DRAM, including initialization, training/calibration, refresh and error reporting as applicable.
- ❑ Implement DMA, queues, arbitration, response tracking and backpressure.
- ❑ Double-buffer transfers where useful and budget all in-flight data.
- ❑ Provide scale/metadata bandwidth alongside weight bandwidth.
- ❑ Define read/write collision behavior and map it to the selected macros.
- ❑ Add appropriate error detection/correction, test, repair or redundancy.

A realistic integration boundary



The present asynchronous array reads and broad access patterns are not a complete SRAM interface. A new latency-aware interface is required before claiming a memory macro can simply replace an array. A full Q16.16 image for the current 1B model would consume roughly 4.94 GB, before dynamic state.

Deliverable: Memory architecture, address map, controller/PHY integration and macro contracts.

Completion evidence: The complete workload runs with realistic latency, stalls and capacity limits; no ideal zero-latency memory assumption remains hidden.

9 Size model storage, KV capacity and bandwidth

For P stored parameters at q bits each, bare weight storage is

$$M_W = \frac{Pq}{8} \text{ bytes,}$$

before scales, zero points, alignment, replication, repair and error protection.

Stored parameters	8-bit	4-bit
1 billion	1 GB	0.5 GB
8 billion	8 GB	4 GB
70 billion	70 GB	35 GB
405 billion	405 GB	202.5 GB
1 trillion	1 TB	500 GB

These are decimal units and hypothetical precision choices, not a claim that all checkpoints preserve acceptable quality at four bits.

For conventional MHA/GQA with uniform dimensions and cache precision,

$$M_{KV} = 2BLTH_{KV}D_h b.$$

B is concurrent sequences, L layers, T cached tokens, H_{KV} KV heads, D_h head size and b bytes per cached element. The factor two accounts for K and V. For the local model ($L = 16$, $H_{KV} = 8$, $D_h = 64$), an FP16 cache requires 256 MiB at 8,192 tokens or 4 GiB at 131,072 tokens *per conversation*. The present 32-bit cache representation doubles those amounts. MLA and recurrent architectures need different storage equations.

For dense batch-one decode with one full external matrix-weight read per token,

$$BW_W \gtrsim M_W R,$$

where R is tokens/s. A hypothetical 70B, 4-bit, 100-token/s target requires at least 3.5 TB/s for bare weights alone. Its dense projection work is roughly $2P$ operations/token (multiply and add counted separately), or 14 TOP/s in the implemented number format, excluding attention and nonlinearities.

Batching can amortize weight reads; local weights change where bandwidth must be provided. Neither estimate is an achieved rate. Actual latency is bounded by compute, memory, communication and dependencies, with imperfect overlap.

Deliverable: A capacity/bandwidth worksheet for every supported workload.

Completion evidence: Every claimed rate fits sustainable hardware budgets with explicit utilization and overhead assumptions.

10 Manage KV state, requests and long contexts

- ❑ Track request ownership, model identity, sequence length and token position.
- ❑ Define per-layer/head cache layouts and append/read ordering.
- ❑ Check capacity before acceptance; report full-context errors explicitly.
- ❑ Allocate, release and reuse storage without cross-request reads.
- ❑ Invalidate incompatible caches when the model, adapter or relevant execution configuration changes.
- ❑ Define reset, cancel, abort, retry and context-restart behavior.
- ❑ Introduce paging or block allocation if required for long contexts and concurrency.
- ❑ Add eviction/sliding-window behavior only when consistent with the model and product promise.
- ❑ Qualify KV quantization independently of weight quantization.
- ❑ Define ownership/reference counting for shared prefixes if supported.
- ❑ Budget cache read bandwidth as context grows, not just storage bytes.
- ❑ Enforce access isolation and any required erasure policy.

Production scheduling

Continuous batching, chunked prompt processing and admission control can improve utilization, but add request IDs, synchronization and resource-management logic. Decide whether prompt and decode work share a device or use separate resources; the latter also requires KV transfer and coordination.

Current cache behavior: `clear_cache` resets the valid position; it does not overwrite every KV bit. Logical invalidation can support dense SRAM mapping, but it is not physical data erasure. The integration must prevent stale entries from being read or exposed.

Deliverable: Cache manager, allocation policy and request-lifecycle specification.

Completion evidence: Tests cover full capacity, reuse, model changes, cancellations, multiple users and long-context numerical behavior with no stale or cross-request state.

11 Make execution programmable and observable

Changing RTL parameters changes elaborated hardware. Runtime model portability requires a supported instruction set, descriptors or microcode plus reloadable weights.

Descriptor field	Required meaning
Opcode and version	Supported operation and compatible command format.
Tensor references	Source/destination bases, dimensions, strides and bounds.
Numerics	Formats, scale locations, accumulation and output rules.
Dependencies	Ordering, barriers, events and buffer lifetime.
Request identity	Model/context ownership and completion destination.
Status	Completion, error reason, progress and performance counters.

- ☐ Latch accepted commands; use explicit ready/valid or equivalent handshakes.
- ☐ Check opcodes, formats, addresses and dimensions before issuing accesses.
- ☐ Define backpressure, queue depth, outstanding operations and completion ordering.
- ☐ Provide timeouts, watchdogs, cancellation and recovery from invalid state.
- ☐ Define priorities for simultaneous reset, cache clear, configuration and start.
- ☐ Protect model loading against inference using partially written parameters.
- ☐ Expose model/firmware IDs, cycle counts, bytes moved, engine utilization and stall reasons.
- ☐ Add limited trace or diagnostic readback to locate the first incorrect tensor.
- ☐ Provide output backpressure or a documented bounded-buffer protocol.

A hardware sequencer may be an FSM, microcoded controller, or management CPU issuing accelerator commands. The constraint is that the accelerator computes the transformer tensors; it does not prohibit software that schedules operations.

Deliverable: Command/register specification, sequencer, trace facilities and conformance tests.

Completion evidence: Legal programs execute correctly under stalls; invalid requests fail predictably without corrupting another request.

12 Support different families and very large models

Model feature	Additional support to plan
GPT-style variants	LayerNorm, biases, GELU variants, learned or alternative positions.
Attention variants	MHA/MQA/GQA layout rules, masks, windows, position scaling and separate output weights.
Mixture of experts	Router/top-k selection, token dispatch, expert addressing, imbalance handling and weighted output combination.
Latent attention	Its projection graph and compressed cache representation.
Encoder–decoder	Encoder computation, cross-attention and encoder-state storage.
Vision/audio	Required frontend operators, preprocessing, encoder and cross-modal integration.
Recurrent/state-space	Architecture-specific state updates, scans, convolution or other operators.
Adapters	Storage, low-rank projections, selection, model/cache compatibility and quality tests.

The support matrix must specify subsets rather than promise all of these at once. DeepSeek-V3, for example, documents 671B total parameters, 37B active per token, MoE and MLA [3]. A dense Llama loop with larger parameters does not implement that graph. Active parameters reduce some work per token; total expert weights still require a storage strategy.

Beyond a single chip

- Select tensor, pipeline and/or expert parallelism; generate partitions in the compiler.
- Budget link latency/bandwidth and collectives: reductions, gathers and routed expert traffic.
- Implement physical links, packet queues, flow control, synchronization and integrity checks.
- Define topology, retry/failure behavior and deadlock prevention.
- Coordinate distributed model loading, request state, completion and recovery.
- Budget package/board/rack power, cooling and network capacity at the system level.

Deliverable: A bounded compatibility matrix and, if needed, a multi-device execution design.

Completion evidence: A second supported model runs on unchanged hardware; a partitioned model matches its single-system reference and survives communication stalls.

13 Build independent verification and release evidence

Verification should expose incorrect arithmetic and incorrect dataflow, not merely demonstrate that the controller eventually asserts done.

- Establish an independent oracle with specified arithmetic semantics and real checkpoint data.
- Test signed extrema, zeros, tiny values, scale outliers, rounding ties and long reductions.
- Compare Q/K/V projections, norm statistics, RoPE outputs, attention scores, probabilities, FFN intermediates and final logits.
- Test nonidentity rotary angles and long-context position behavior.
- Exercise nonmultiple tiles, supported shape boundaries and overflow in address calculations.
- Inject memory stalls, variable latency, backpressure and interruptions.
- Test reset/cancel mid-operation, full-cache behavior, model updates and request isolation.
- Add assertions for valid state transitions, bounds, ordering and completion.
- Use code/functional coverage and review uncovered behaviors; coverage percentage alone is not correctness.
- Use formal properties for suitable blocks/protocols and equivalence checks across synthesis/ECO steps.
- Run gate-level and timing-aware checks appropriate to the design and tool flow.
- Evaluate model quality over multiple prompts, tasks and context lengths; matching one answer is insufficient.

Preserve a reproducible evidence bundle

Store source revision, model hash, numerical contract, generator seed, tool and IP versions, configuration, expected/observed values, tolerances, logs and result summary. Make regressions run automatically when RTL, compiler, firmware or model packing changes. Record approved exceptions with owner and rationale.

Deliverable: Operator, layer, model, protocol and implementation regression suites.

Completion evidence: A release candidate meets declared quality limits and all required checks; numerical and hardware failures identify a reproducible first point of divergence.

14 Model performance and validate on a hardware prototype

Measure prompt processing and generation separately. A high peak MAC count can coexist with poor token latency if memory, reductions or communication starve it.

- Build a cycle/data-movement model with actual memory ports, transfer sizes, queue depth, dependencies and clock assumptions.
- Estimate area and power from representative mapped blocks and memory macros.
- Measure prefill latency, time to first token, decode latency and aggregate throughput.
- Sweep prompt length, generated length, batch size and request concurrency.
- Record p95/p99 latency and admission/queue behavior where relevant.
- Profile bank conflicts, stalls, arithmetic utilization and bytes transferred.
- Measure chip/card/system power at clearly stated boundaries; calculate energy per token.
- Compare model quality at the same numerical settings used for speed measurements.
- Use an FPGA/emulation prototype with real memory traffic to validate integration.
- Separate prototype clock/resource limits from target ASIC projections.

Minimum benchmark record

Record	Required context
Model and software	Checkpoint hash, compiler/runtime version, precision and quality result.
Workload	Prompt/output lengths, context, sampling policy, concurrency and warm/cold state.
Hardware	Devices, memory, clocks, cooling and measurement method.
Performance	First-token latency, inter-token latency, throughput, variability and repetitions.
Energy/cost	Power boundary, energy per token, utilization and cost assumptions.

MLPerf provides a reference for defined inference workloads and quality requirements [4]. Do not claim formal benchmark compliance without following the applicable rules.

Deliverable: A measured prototype report and revised capacity/area/power budget.

Completion evidence: The claimed workload meets its targets with no hidden host tensor computation or unreported precision/quality change.

15 Integrate the chip platform and obtain technology IP

Digital platform

- ☐ Select host transport, register map, DMA, command queues and interrupts.
- ☐ Integrate boot/configuration control and firmware storage where needed.
- ☐ Design clock/reset distribution and verify clock-domain/reset-domain crossings.
- ☐ Add clock gating, power management and valid power-state transitions.
- ☐ Provide watchdogs, debug access, performance counters and error reporting.
- ☐ Add temperature/voltage monitoring and controlled thermal throttling where required.
- ☐ Protect host/DMA address ranges and isolate requests appropriate to the deployment.
- ☐ Define model/firmware integrity, update and rollback requirements.

Foundry and licensed implementation inputs

- ☐ Select foundry, process, approved design kit, standard cells and sign-off flow.
- ☐ Obtain SRAM/ROM macros with the required size, banking, ports and characterized corners.
- ☐ Obtain I/O, ESD, level-shifter/isolation and power-domain cells as needed.
- ☐ Obtain clock-generation IP if the chip needs an internal PLL/oscillator.
- ☐ Obtain memory controller/PHY and high-speed interface IP for selected external interfaces.
- ☐ Confirm physical, timing, electrical, functional and test views are mutually compatible.
- ☐ Resolve license/manufacturing rights, support, integration constraints and package requirements.

Do not equate an open PDK installation with an agreed commercial production flow. The public SKY130 documentation describes the status and limitations of its release [5]; confirm accepted tools, decks and qualification with the actual fabrication partner. A process node alone does not establish that the required memory or interface IP is available.

Deliverable: System specification, IP inventory, technology agreement and integration models.

Completion evidence: All hard macros and interfaces have usable views, verified integration and a path accepted by the fabrication partner.

16 Close physical design and prepare a tapeout package

- ❑ Establish floorplan, hierarchy, macro placement, pin/bump constraints and routing channels.
- ❑ Design the power grid using estimated current, package delivery and decoupling needs.
- ❑ Write complete clock, I/O, uncertainty, load and timing-exception constraints.
- ❑ Map logic to the chosen libraries; insert appropriate test and power structures.
- ❑ Place cells, construct clock trees and complete detailed routing.
- ❑ Extract parasitics and close setup/hold timing across required modes and corners.
- ❑ Check clock quality, unconstrained paths, signal integrity and crosstalk as applicable.
- ❑ Analyze static/dynamic voltage drop and electromigration with justified activity assumptions.
- ❑ Complete DRC, LVS, electrical, antenna, density and manufacturing-rule checks.
- ❑ Add required fill, seal ring, die identification, pads/bumps and package connectivity.
- ❑ Recheck equivalence and timing after engineering changes.
- ❑ Release GDS/OASIS and required netlists, abstracts, constraints, reports and checksums.

Define acceptable closure with the foundry

The goal is no unapproved violations under the selected sign-off rules. Any waiver must be reviewed and accepted by the responsible design/fabrication authority; a script returning successfully is not an engineering waiver.

The older one-row SKY130 experiment does not predict full-decoder area by a simple multiplier. Memory topology, cell implementation, routing and clock distribution change with architecture. Nor does the new generic structural target establish the absence of combinational mapping problems.

Deliverable: A reviewed tapeout package and signed release checklist.

Completion evidence: The exact released design passes required checks, uses accepted IP views and matches the package/manufacturing submission rules.

17 Design production test and silicon reliability

- Develop a design-for-test plan before physical implementation is finalized.
- Insert scan/test access and generate automated fault patterns as appropriate.
- Cover stuck-at and timing-related faults with measurable targets and justified exclusions.
- Add SRAM self-test, repair/redundancy and ROM verification where supported.
- Plan boundary scan, test clocks, reset control and power modes.
- Create wafer-probe and packaged-part test flows, fixtures and program versions.
- Define pass/fail bins, test limits, test time and traceability requirements.
- Characterize voltage/frequency/temperature operating margins.
- Qualify ESD, latch-up, package integrity and lifetime behavior for the target market.
- Plan fault injection, field error reporting and recoverable failure behavior.
- Establish yield analysis, failure analysis and a feedback process for revisions.

First-silicon bring-up order

1. Verify rails, current draw, clock and reset using safe initial conditions.
2. Establish debug/register access and identify chip/firmware versions.
3. Exercise SRAM, ROM and external-memory initialization/test.
4. Check arithmetic units and small known tensor transfers.
5. Run a known layer, then a full model, against stored reference outputs.
6. Sweep workloads, clocks, voltage/temperature conditions and interfaces within approved limits.

Manufacturing yield and reliability are separate from neural-network numerical quality. A design must satisfy both, with a test policy that can distinguish silicon defects from software or model-image failures.

Deliverable: DFT architecture, test patterns/programs, characterization and qualification plan.

Completion evidence: Working parts can be identified repeatably, faulty parts rejected and field failures traced to a reproducible configuration.

18 Build the package, board, cooling and service software

Physical product

- Select package technology, substrate, pin/bump assignment and escape routing.
- Co-design power/ground delivery, signal routing and thermal paths.
- Create PCB schematics, stack-up, impedance constraints and layout.
- Specify regulators, rail sequencing, protection, decoupling and clock sources.
- Route external memory and host interfaces with signal/power integrity checks.
- Provide debug/programming connectors, sensors, test points and assembly fixtures.
- Design heatsink, airflow, thermal interface, mounting and enclosure.
- Release BOM, fabrication/assembly data and board production tests.

Software a customer can use

- Deliver firmware, driver, model loader and a documented runtime API.
- Validate model/firmware/compiler compatibility and failure reporting.
- Provide tokenization, templates and generation controls promised by the product.
- Decide logits versus token output; argmax supplies greedy decoding only.
- Implement temperature, top-k/top-p, penalties and RNG if supported; identify whether sampling runs on host or accelerator.
- Support request admission, scheduling, cancellation and memory accounting.
- Deliver installation packages, examples, diagnostics, monitoring and profiling.
- Establish signed/versioned releases as appropriate, update/rollback procedures and support policy.

The host may tokenize and serve requests while the accelerator owns the complete transformer forward pass. Make that boundary observable in tests and product documentation so users can distinguish hardware execution from reference software.

Deliverable: Packaged device, reference board and installable software release.

Completion evidence: A new user can install the system, load a supported model, run inference and diagnose documented errors without developer-only intervention.

19 Resolve commercial, licensing and compliance work

- Review checkpoint redistribution/commercial terms, notices, branding and permitted uses.
- Review third-party software, RTL, EDA and hard-IP licenses, including manufacturing rights.
- Review implementation IP/patent exposure with appropriate specialists.
- Determine product/export classification, destinations and end-user requirements.
- Identify applicable EMC, safety and environmental obligations for the final product and markets.
- Define privacy/data-handling obligations for any hosted service.
- Establish warranty, returns, support, service-level and liability terms.
- Secure foundry, assembly/test, memory, board and cooling suppliers with realistic lead times.
- Plan qualification, production ramps, lot traceability, inventory and revision management.

Embedding weights in silicon does not remove their license conditions. Meta’s Llama 3.2 license includes obligations for redistribution/products containing the materials [6]. EU manufacturers must identify applicable legislation and perform the relevant conformity process [7]. Advanced computing export requirements depend on specifications, jurisdiction and destination; review current rules for the selected product [8].

Fund a shipped product, including revisions

Obtain quotations for engineering, EDA/compute, memory/interface IP, masks, prototype runs, wafers, yield, packaging, test, boards, memory, cooling, qualification and support. Include rework/respin contingency. Distinguish prototype shuttle cost from production economics.

$$C_{\text{shipped}} \approx C_{\text{good die}} + C_{\text{package/test}} + C_{\text{board/memory/cooling}} + C_{\text{assembly/support}} + \frac{C_{\text{development}}}{N_{\text{shipped}}}.$$

Assign owners for architecture, numerics, RTL, verification, physical design, DFT, compiler/runtime, board design and manufacturing. These roles may be partners or staff; no universal team size or fabrication price is assumed here.

Deliverable: Commercial plan, supplier/IP agreements and compliance responsibility matrix.

Completion evidence: The product has a viable cost model and a documented route to legal, repeatable manufacture and sale in its selected markets.

20 Execute the project in evidence-based stages

Gate	Work package	Exit evidence
G0	Freeze scope and baseline	Product limits, model/format choices, source snapshot and claim register.
G1	Arithmetic/protocol re-pair	Independent signed arithmetic, nonlinear, bounds and handshake regressions.
G2	Real checkpoint layer	Hardware computes norm, QKV, RoPE, cache, attention and FFN with validated intermediate tensors.
G3	Full real model	All layers and final logits execute through hardware across representative prompts and contexts.
G4	Runtime portability	A second supported checkpoint/configuration runs on unchanged hardware using the importer and schedule.
G5	Realistic memory	Same execution succeeds with memory latency, bandwidth, stalls, capacity and error behavior represented.
G6	Performance prototype	Measured quality, throughput, latency and power/resource evidence against product targets.
G7	Multi-device scale, if needed	Correct partitioned execution, communication budget and recovery behavior.
G8	Tapeout readiness	Technology/IP, DFT, mapped design, physical closure and accepted sign-off package.
G9	First silicon	Packaged parts pass bring-up, interfaces, memory, tensor and model tests.
G10	Product release	Qualification, manufacturing test, usable software, support and commercial obligations complete.

Start with these concrete actions

1. Record the current sources and existing test scope; stop treating the tiny argmax smoke test as evidence of real-model correctness.
2. Choose a first supported checkpoint, context limit and numerical contract. Write focused regressions for the arithmetic/protocol risks in Section 4.
3. Define a latency-aware model-memory interface and a checkpoint packer that matches it.
4. Validate one real layer tensor by tensor, then all layers, then a second model.
5. Obtain representative memory/IP data early and use it to constrain architecture before committing to expensive physical implementation.

These are dependency gates, not calendar estimates. IP availability, numerical results and prototype measurements can require architectural revisions.

21 Maintain a release checklist and risk register

- ☐ Supported-model/operator matrix is published and versioned.
- ☐ Trained weights and executable image are hashed and traceable.
- ☐ Quantization and numerical quality meet predeclared thresholds.
- ☐ No CPU tensor result contributes to the advertised hardware forward pass.
- ☐ Real-memory latency/bandwidth and error behavior are represented in verification.
- ☐ Context, shape, address and concurrency limits are enforced.
- ☐ A second model demonstrates portability if portability is advertised.
- ☐ Required timing, equivalence, physical and electrical checks pass.
- ☐ Manufacturing test and qualification meet agreed targets.
- ☐ Package, board, firmware, driver and model loader versions are compatible.
- ☐ Performance claims include workload, quality and measurement boundaries.
- ☐ Suppliers, licenses, compliance owners and support procedures are assigned.

Track risks as decisions, not vague concerns

Risk		Early evidence	Possible response
Numerical quality		Layer/logit error and task evaluation	Revise formats, accumulators or approximations.
Memory feasibility		Capacity/ports/area/bandwidth from actual macros	Change banking, capacity, external-memory or partition plan.
Timing/congestion		Representative mapped/routed blocks	Pipeline, simplify interfaces, repartition or revise clock target.
Poor utilization		Counters and traffic model	Revise dataflow, buffering, batch policy or lane count.
Model obsolescence		Update policy and customer workload	Favor reloadable weights or limit fixed-model commitments.
Yield/cost		Supplier quotes, test plans and die/package estimates	Revise die size, redundancy, packaging, volume or product target.

Give each entry an owner, evidence link, next experiment, decision deadline and impact on scope. A prototype result should update the plan rather than become an unsupported product claim.

22 Repository map and commands

Paths below are relative to `/Users/computer/dev/verilog`. They identify the current prototype, not a promise that every proposed subsystem already exists.

Path	Purpose
<code>rtl/llama_decode_core.sv</code>	Serial Q16.16 Llama-style decoder prototype.
<code>tb/tb_llama_decode_core.sv</code>	Tiny two-layer/two-token synthetic smoke test.
<code>asic/llama.mk</code>	Decoder generic-lowering and older frozen-tile targets.
<code>asic/llama_frozen_tile_top.sv</code>	Selected fixed-weight Q8 projection tile.
<code>rtl/q8_0_dot_pipeline.sv</code>	Pipelined signed block-dot arithmetic for the frozen tile.
<code>tools/freeze_llama.py</code>	Frozen image/selected-tile preparation.
<code>tools/llama_frozen_reference.py</code>	CPU reference and selected-tile hybrid execution.
<code>asic/LLAMA_RTL_CORE.md</code>	Prototype graph and numerical notes.
<code>asic/LLAMA_FROZEN.md</code>	Older frozen tile, co-simulation and physical experiment.

Existing local checks

```
cd /Users/computer/dev/verilog
make -C asic llama-core-sim
make -C asic llama-core-synth
```

The first command runs the synthetic test. The second performs generic lowering with memories preserved; it is not full standard-cell mapping or timing closure. Previously recorded smoke output was 328 and 356 cycles for the two tokens. Those numbers do not describe Llama 1B inference speed.

Rebuild this PDF

```
cd /Users/computer/dev/verilog/output/pdf
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  inference_chip_product_roadmap.tex
```

Use a TeX installation containing the packages declared in the source. This document is standalone and does not read the model weights to compile. The LaTeX source remains editable alongside the PDF.

23 Glossary

Term	Meaning in this project
ASIC	Integrated circuit designed for a particular workload or family of workloads.
RTL	Clocked logic description used to model and synthesize digital hardware.
GEMV / GEMM	Matrix–vector / matrix–matrix multiplication.
MAC	Multiply–accumulate operation or the circuit performing it.
Prefill / decode	Process prompt tokens / generate subsequent tokens incrementally.
KV cache	Mutable keys and values retained from previous attention positions.
GQA / MQA	Several query heads share groups of KV heads / one KV head.
MoE / MLA	Mixture of experts / multi-head latent attention.
Quantization	Represent values using fewer bits with defined scales or exponents.
Q16.16	Signed 32-bit fixed point with 16 fractional bits in this prototype.
SRAM / ROM	Writable static memory / read-only memory; physical macros differ from RTL arrays.
DMA / PHY	Autonomous memory transfer engine / physical electrical interface.
PDK	Process design kit: technology models, rules and implementation data.
STA / PVT	Static timing analysis / process, voltage and temperature conditions.
DRC / LVS	Design-rule geometry checks / layout-versus-schematic connectivity comparison.
IR drop / EM	Supply-voltage loss / electromigration from current stress.
CDC / RDC	Clock-domain crossing / reset-domain crossing.
DFT / ATPG	Design for test / automatic test-pattern generation.
MBIST / ECC	Memory built-in self-test / error-correcting code.
MPW / OSAT	Shared multi-project wafer / outsourced semiconductor assembly and test.
NRE / ECO	Non-recurring engineering cost / engineering change order.
GDSII / OASIS	Layout exchange formats used in the manufacturing handoff.
TTFT / p99	Time to first token / latency below which 99% of observed requests fall.

References

- [1] Taalas, *The path to ubiquitous AI*. Vendor description of model specialization and storage/compute integration; not an open circuit specification. <https://taalas.com/the-path-to-ubiquitous-ai/>
- [2] NVIDIA, *TensorRT-LLM: Multi-Head, Multi-Query, and Group-Query Attention*. Examples of attention, KV-cache and serving techniques; not the RTL specification for this project. <https://nvidia.github.io/TensorRT-LLM/features/attention.html>
- [3] DeepSeek AI, *DeepSeek-V3 official repository*. Architecture example documenting total/active parameters, MoE and MLA. <https://github.com/deepseek-ai/DeepSeek-V3>
- [4] MLCommons, *MLPerf Inference: Datacenter*. Benchmark methodology and workload/quality context. <https://mlcommons.org/benchmarks/inference-datacenter/>
- [5] SkyWater, *SKY130 PDK documentation*. Technology documentation and release status; confirm production requirements with the selected fabrication partner. <https://skywater-pdk.readthedocs.io/en/main/>
- [6] Meta, *Llama 3.2 Community License Agreement*. Review the applicable checkpoint version and intended distribution. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/LICENSE
- [7] European Commission, *CE marking: Manufacturers*. Determine applicable legislation for the finished product and target market. https://single-market-economy.ec.europa.eu/single-market/goods/ce-marking/manufacturers_en
- [8] U.S. Bureau of Industry and Security, *Export Administration Regulations: Table of Contents*. Requirements must be assessed using current rules, product specifications, jurisdiction and destination. <https://www.bis.gov/regulations/ear/table-of-contents>

Source boundary. Repository observations are based on the local files listed in Section 22; their current contents may change after this snapshot. Numerical sizing examples are calculations under their stated assumptions. External sources were reviewed for this roadmap on 20–21 September 2026. No vendor performance claim is used as a predicted performance figure for this design.